

Corrigé — Contrôle 2nde : Boucle while

Exercice 1 — Suite de Syracuse

1. Fonction complétée :

```

1 def syracuse(n) :
2     while n != 1 :
3         if n % 2 == 0 :           # n est pair
4             n = n // 2
5             print(n)
6         else :                   # n est impair
7             n = 3 * n + 1
8             print(n)

```

► La condition $n \% 2 == 0$ teste la parité. Pair : $n = n // 2$ (division entière). Impair : $n = 3*n + 1$.

2. Avec $n = 6$:

► $6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$
 ► *Détail* : 6 pair $\rightarrow 3$; 3 impair $\rightarrow 10$; 10 pair $\rightarrow 5$; 5 impair $\rightarrow 16$; $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.
 La boucle s'arrête car $n = 1$.

3. Conjecture :

► **Conjecture de Syracuse (Collatz)** : quelle que soit la valeur initiale n choisie, la suite finit toujours par atteindre 1. Cette conjecture est vérifiée pour des milliards de valeurs mais n'est pas encore démontrée mathématiquement.

Exercice 2 — L'escargot

1. Fonction complétée :

```

1 def parcours() :
2     distance = 1
3     jour = 1
4     while distance < 3 :
5         jour = jour + 1
6         distance = distance + 1/jour
7     return jour, distance

```

► Condition : $distance < 3$ — la boucle continue tant que la distance n'atteint pas 3 m.

► Incrément : $1/jour$ — au jour j , l'escargot avance de $\frac{1}{j}$ mètre.

► Résultat : la distance $1 + \frac{1}{2} + \frac{1}{3} + \dots$ dépasse 3 m au bout d'environ **11 jours** ($\sum \approx 3,02$ m).

2. Fonction avec paramètre `seuil` :

```

1 def parcours(seuil) :
2     distance = 1
3     jour = 1
4     while distance < seuil :
5         jour = jour + 1
6         distance = distance + 1/jour
7     return jour, distance

```

► On remplace la valeur fixe 3 par le paramètre `seuil`. Appel : `parcours(5)` pour dépasser 5 m.

Exercice 3 — Tableau d'état

1. Tableau d'état complété :

	Avant	Pass.1	Pass.2	Pass.3	Pass.4	Pass.5	Pass.6	Pass.7
n	0	1	2	3	4	5	6	7
s	0	2	6	12	20	30	42	56

► Calcul : $s = 0 + 2 = 2$; $2 + 4 = 6$; $6 + 6 = 12$; $12 + 8 = 20$; $20 + 10 = 30$; $30 + 12 = 42$; $42 + 14 = 56$.

► Formule : $s = 2(1 + 2 + \dots + n) = n(n + 1)$. Vérification : $7 \times 8 = 56$ ✓

2. La valeur de s qui arrête la boucle est **56** — c'est la première valeur ≥ 50 (au Pass. 7).

3. La première valeur de n telle que $s \geq 50$ est $n = 7$ ($s = 56 \geq 50$, alors qu'au Pass. 6 : $s = 42 < 50$).

Exercice 4 — Distributeur automatique de billets

1. L'instruction de la ligne 2 (`if n % 10 != 0`) vérifie que le montant demandé est un **multiple de 10 €**. Si ce n'est pas le cas, le DAB ne peut pas le distribuer avec ses billets de 10 € et 20 € → retourne 'impossible'.

2. Pour un retrait de 330 € :

► $330 \% 10 = 0 \rightarrow$ montant valide.

► La boucle soustrait 20 € à chaque tour tant que $n \geq 20$: $330 \rightarrow 310 \rightarrow \dots \rightarrow 10$ (en 16 tours).

► Après la boucle : $n = 10$, $i = 16$.

► `billets_20 = 16`, `billets_10 = 10 // 10 = 1`.

► Vérification : $16 \times 20 + 1 \times 10 = 320 + 10 = 330$ € ✓

Exercice 5 — Algorithme mystère (Bonus)

1. Tableau complété :

x	y	Résultat
2	5	10 = 2×5
2	100	200 = 2×100
3	15	45 = 3×15
8	0,01	0,08 = $8 \times 0,01$
5	2,3	11,5 = $5 \times 2,3$

► Trace (2, 5) : $x = 2$ pair $\rightarrow x = 1, y = 10$; $x = 1$ impair $\rightarrow x = 0, z = 10$. Retourne $10 = 2 \times 5$.

► Trace (3, 15) : $x = 3$ impair $\rightarrow x = 2, z = 15$; $x = 2$ pair $\rightarrow x = 1, y = 30$; $x = 1$ impair $\rightarrow x = 0, z = 15 + 30 = 45 = 3 \times 15$.

2. Cette fonction calcule le **produit** $x \times y$. C'est la **multiplication russe** (algorithme par doublement) : à chaque étape, si x est pair on divise x par 2 et on double y ; si x est impair on accumule y dans z . La somme des y ajoutés vaut $x \times y$ initial.